

بوت کمپ DevOps2

سیستم مدیریت وظایف (Task Management System)

۱. مقدمه و اهداف پروژه

این پروژه شامل راه اندازی، استقرار و مدیریت زیرساخت یک سیستم مدیریت وظایف (Task Manager) است. سوره کدهای سمت **Backend** و **Frontend** به صورت کامل در اختیار شما قرار می گیرد. هدف از این کار، تمرکز صددرصدی شما بر روی مفاهیم، ابزارها و وظایف یک مهندس DevOps است تا بدون درگیری با توسعه نرم افزار، یک چرخه حیات کامل (Full DevOps Lifecycle) را در یک محیط عملیاتی (Production-Ready) پیاده سازی کنید. (هرچند قابلیت تغییر آن وجود دارد و شامل امتیاز است)

۲. معماری نرم افزار (Application Architecture)

سوره کدهایی که در اختیار شما قرار گرفته اند دارای معماری زیر هستند:

- **Backend:** توسعه یافته با فریمورک (Python) FastAPI. دارای ارتباط با دیتابیس رابطه ای، لایه کش، و دارای (/health) جهت بررسی وضعیت سرویس.
- **Database:** استفاده از PostgreSQL برای ذخیره سازی داده های کاربران و تسک ها.
- **Cache:** استفاده از Redis جهت مدیریت نشست ها و بهینه سازی درخواست ها.
- **Frontend:** توسعه یافته با React. ارتباط با بک اند از طریق مسیر api/tasks.

۳. فازهای اجرایی و الزامات اصلی پروژه (Core Requirements)

فاز اول: (Dockerization)

شما باید برای سرویس های بک اند و فرانت اند فایل های Dockerfile استاندارد بنویسید (فایل فرانت اند به عنوان نمونه در اختیار شماست). رعایت موارد زیر الزامی است:

- استفاده از رویکرد Multi-stage Build برای کاهش حجم ایمیج ها (به ویژه در فرانت اند با Nginx).

- استفاده از ایمج‌های پایه (مثل Alpine).
- رعایت اصول امنیتی داکر از جمله اجرای کانتینر با دسترسی محدود (Non-root user).
- تعبیه HEALTHCHECK درون فایل‌های داکر.
- نوشتن docker-compose.yml جامع برای اجرای یکپارچه تمامی سرویس‌ها (Backend, Frontend, Postgres, Redis) در محیط Local.

فاز دوم: کنترل نسخه و CI/CD

- Git Strategy: پیاده‌سازی ساختار Branching استاندارد (شامل main، develop و feature).
- GitLab CI/CD: طراحی پایپ‌لاین خودکار شامل مراحل:
 ۱. Test/Lint: بررسی اولیه کدها.
 ۲. Build: ساخت ایمج‌های داکر.
 ۳. Push: ارسال ایمج‌ها به Docker Registry خصوصی یا عمومی.
 ۴. Update Git: به‌روزرسانی تگ ایمج‌ها در فایل values.yml مخزن Git پس از موفقیت مراحل قبل.
- Argo CD: اتصال به مخزن Git حاوی Helm Chart و فعال‌سازی همگام‌سازی خودکار برای استقرار اپلیکیشن و اعمال نسخه‌های جدید در کلاستر Kubernetes.

فاز سوم: استقرار در Kubernetes

مدیریت کانتینرها باید به صورت درست در کوبرنیتیز پیاده سازی شود:

- Deployments: ایجاد دیپلویمنت ها با حداقل ۲ نسخه (Replica=2) برای تضمین High Availability.
- Probes: تنظیم دقیق Liveness Probe (با استفاده از مسیر /health) و Readiness Probe.
- Resource Management: تعیین دقیق Limits و Requests برای CPU و RAM پادها.
- ConfigMaps & Secrets: جلوگیری از Hardcode شدن متغیرها. رمز عبور دیتابیس (DATABASE_URL)، اطلاعات ردیس (REDIS_HOST) و سایر موارد حساس باید تماماً از طریق Kubernetes Secrets به پادها تزریق شوند.
- Rolling Update: تنظیم استراتژی به روزرسانی برای رسیدن به Zero Downtime.
- Helm: تهیه Helm Chart برای سرویس های اپلیکیشن، تعریف پارامترهای استقرار در values.yaml و نگهداری Chart و مقادیر غیرحساس در Git. استقرار و به روزرسانی اپلیکیشن با استفاده از این Chart و از طریق Argo CD انجام شود.

فاز چهارم: Logging, Monitoring و زیرساخت

- Prometheus: پیاده سازی سیستم مانیتورینگ برای پایش وضعیت کلاستر، میزان مصرف منابع، و شناسایی پادهای دارای مشکل (مانند وضعیت CrashLoopBackOff).
- Ansible (IaC): خودکارسازی آماده سازی سرورها (Provisioning). نوشتن Playbookهایی برای نصب ابزارهای پایه، نصب Docker و تنظیمات اولیه کلاستر.

۴. بخش امتیازی (Bonus)

گروه هایی که علاوه بر نیازمندی های اصلی، بتوانند یک یا چند مورد از تکنولوژی های زیر را در پروژه خود پیاده سازی و ارائه کنند، از امتیاز ویژه و بالاتری برخوردار خواهند شد. برخی از این موارد نیازمند تحقیق و یادگیری فردی (Self-study) است:

دریافت امتیاز هر بند منوط به راه اندازی صحیح و نمایش عملی عملکرد آن است.

Cluster deployment, orchestration, and management

۱. CRD (Custom Resource Definition): ایجاد و استفاده از ریسورس های اختصاصی (Custom) برای استقرار اجزای سیستم در کوبرنیتیس.

۲. Rancher: نصب و استفاده از این پلتفرم جهت مدیریت کلاستر کوبرنیتیس و دیپلوی کردن اپلیکیشن از طریق آن.

۳. RKE2 + Cilium + Envoy Gateway: راه اندازی کلاستر Kubernetes با RKE2، انتخاب Cilium به عنوان CNI و استفاده از Envoy Gateway برای پیاده سازی Gateway API. مسیرهای دسترسی به Frontend و API از طریق منابع Gateway و HTTPRoute تعریف و عملکرد آنها نمایش داده شود.

مقیاس پذیری و شبکه (Networking & Scaling)

۴. HPA (Horizontal Pod Autoscaler): پیاده سازی Auto-scaling خودکار برای افزایش یا کاهش تعداد پادها بر اساس بار ترافیکی (CPU/Memory).

۵. VPA (Vertical Pod Autoscaler): پیاده سازی Auto-scaling عمودی برای تخصیص و تغییر خودکار منابع (Limits/Requests) پادها.

۶. Traefik Ingress Controller: استفاده از Traefik به عنوان Ingress Controller جهت مدیریت ترافیک ورودی به کلاستر (به جای Nginx Ingress).

Monitoring, logging and observability (تحقیق آزاد)

۷. ELK Stack: راه اندازی Elasticsearch, Logstash, Kibana برای تجمیع و مدیریت لاگ ها (Log Management).

۸. Loki: راه اندازی Grafana Loki جهت ردیابی، تریس و مدیریت سبک لاگ ها در محیط کوبرنیتیز (آموزش داده نشده).

۹. VictoriaMetrics: پیاده سازی و استفاده از VictoriaMetrics به جای Prometheus برای مانیتورینگ و نگهداری متریک ها.

مدیریت ذخیره سازی پیشرفته (تحقیق آزاد – آموزش داده نشده)

۱۰. Rook Ceph / Ceph: راه اندازی Ceph به کمک اپراتور Rook جهت مانت کردن Volume ها و استفاده از آن به عنوان Dynamic Provisioning در کلاستر کوبرنیتیز. این بخش مهارت فوق العاده ای در حوزه Storage به اثبات می رساند.

پشتیبان گیری و بازیابی

۱۱. Velero: راه اندازی پشتیبان گیری از منابع Kubernetes و داده های پایدار اپلیکیشن و تعیین محل ذخیره پشتیبان ها. روش پشتیبان گیری از Volume ها متناسب با ذخیره ساز انتخاب شود. بازیابی در یک محیط آزمایشی انجام و بازگشت سرویس و داده های نمونه PostgreSQL پس از Restore بررسی و نمایش داده شود.

کیفیت و امنیت کد

۱۲. SonarQube: راه اندازی ابزار و اتصال آن به GitLab CI برای تحلیل استاتیک کدهای Backend و Frontend و ارائه گزارش کیفیت و امنیت کد. یک Quality Gate تعریف شود تا در صورت عدم پذیرش نتیجه تحلیل، پایپ لاین پیش از به روزرسانی تنظیمات استقرار در Git متوقف شود.

۵. سناریوی جلسه داوری (Demo Scenarios)

در جلسه داوری، علاوه بر بررسی کدها و زیرساخت، اجرای زنده (Live Demo) اپلیکیشن و بررسی الزامات اصلی پروژه ضروری است.